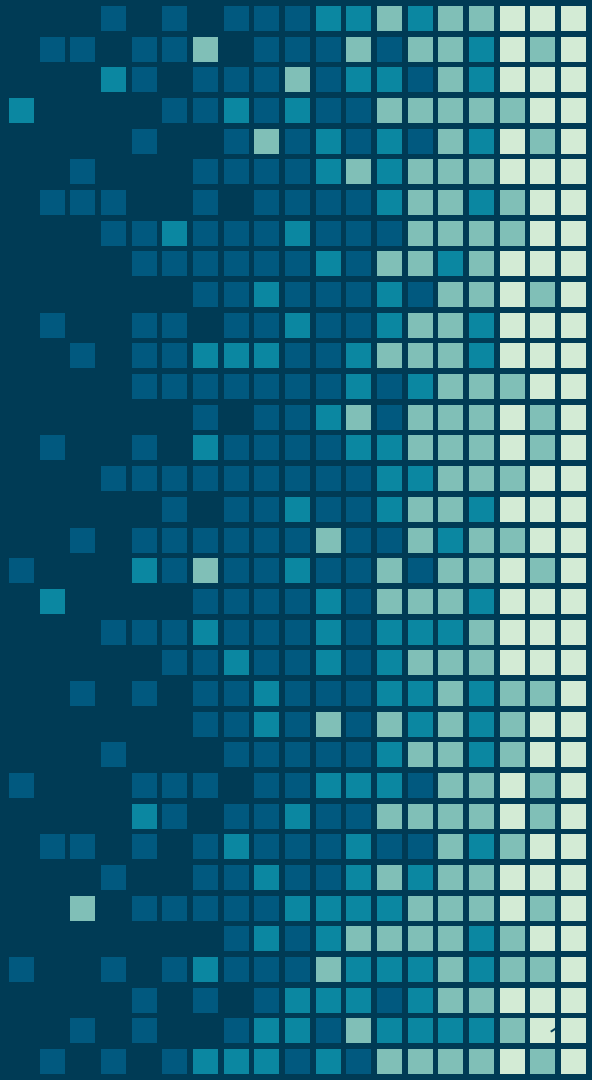
EMBEDDED CONTROL

Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



Reading from Encoders to estimate position and velocity

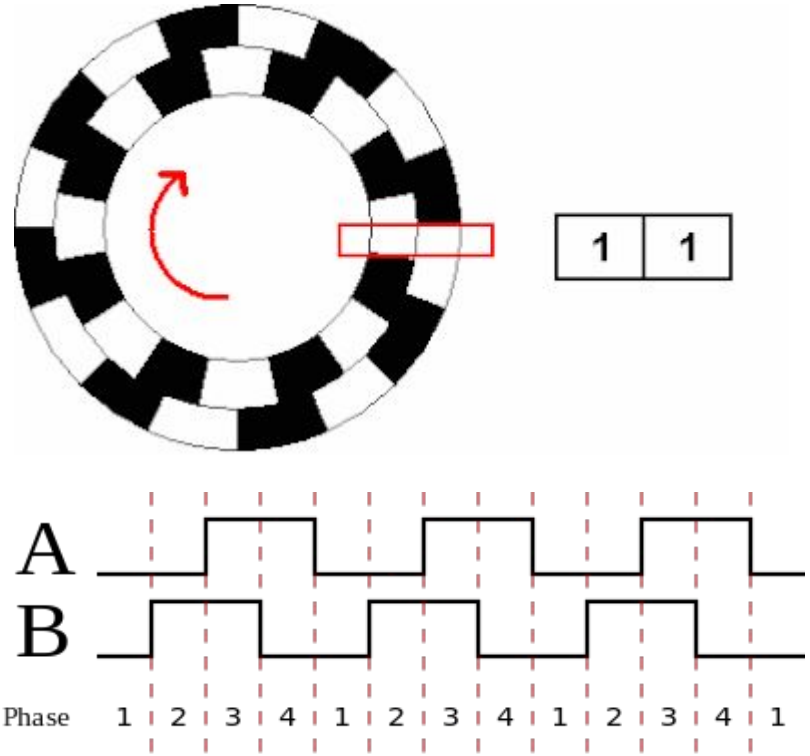
Encoders

- Encoders are used to sense rotations of a shaft
- They transduce (encode) an analog angular displacement to digital signals
- Different constructive technologies (beyond the scope of this course!): optical, hall effect, mechanical, etc
- Two different types:
  - **Absolute**: they associate the position with a code. When turned off/on they have "memory" of the position
  - **Relative**: they sense changes of the position. When turned on/off they lose track of the position

Incremental encoders

- Very popular, used to sense angles and angular speeds
- Resolution: the minimum angular displacement the encoder can sense
- Resolution measured in “counts per revolution” (CPR)
- CPRs can go up to the order of 10000
- Notice: too high resolution is not the best choice to sense high angular speeds (why?)
- Regardless of the technology used: two channels are used to sense “counts” (or ticks) and direction of rotation.

Quadrature Encoders (optical example)



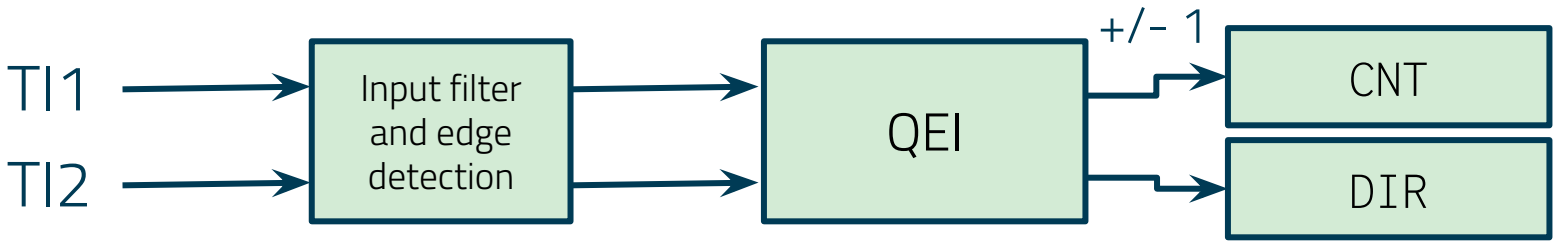
source: wikipedia

- Two channels are used
- Two LEDs and photodiodes used to sense movement
- When the light passes we have a logic 1
- The two channels are square waves in quadrature (phase difference +/- 90°)
- Sign of the phase shift indicates the rotation direction, depending on wiring and polarity.

How to decode the encoder?

- STM32 MCUs have quadrature encoder interfaces (QEI) for most general purpose timers
- QEIs help decoding the two quadrature channels TI1 TI2 (channels are called this way by ST)
- We employ the Timer counter to:
  - count up when a clockwise displacement is sensed
  - count down when a counterclockwise displacement is sensed
- The angular position is then "encoded" in the `CNT` register!
- The direction is stored in the `DIR` timer's register (`DIR=0` counting up, `DIR=1` counting down)

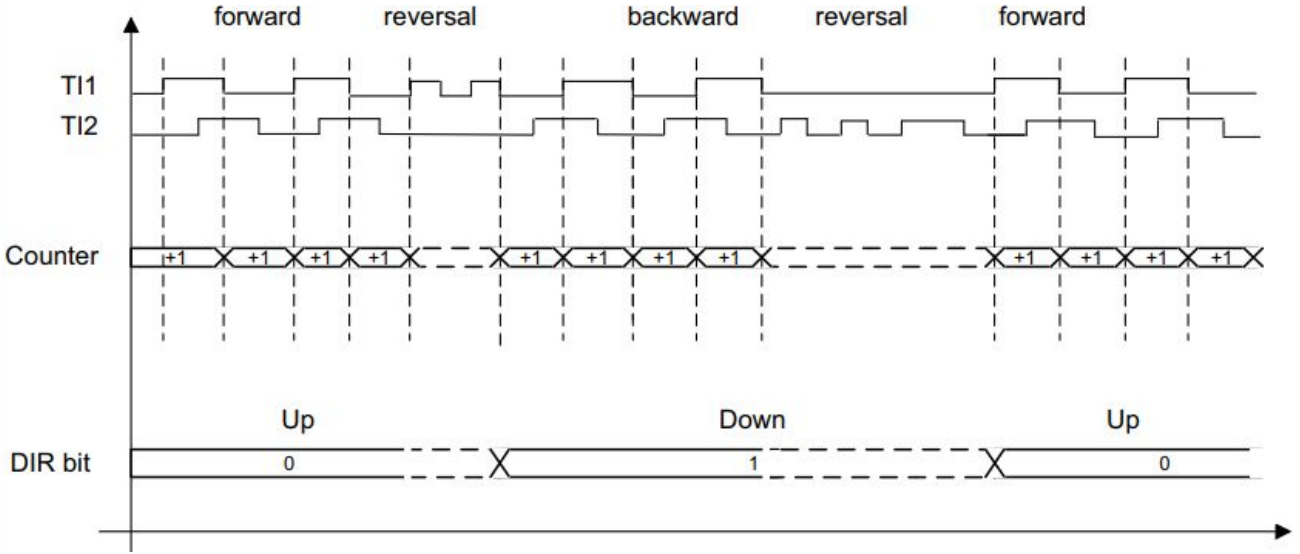
QEI structure



Counting resolution

Two resolutions:

- **2x**: count up or down by 1 at rising or falling edge of TI1 OR TI2 channel (in the example TI1 is used)

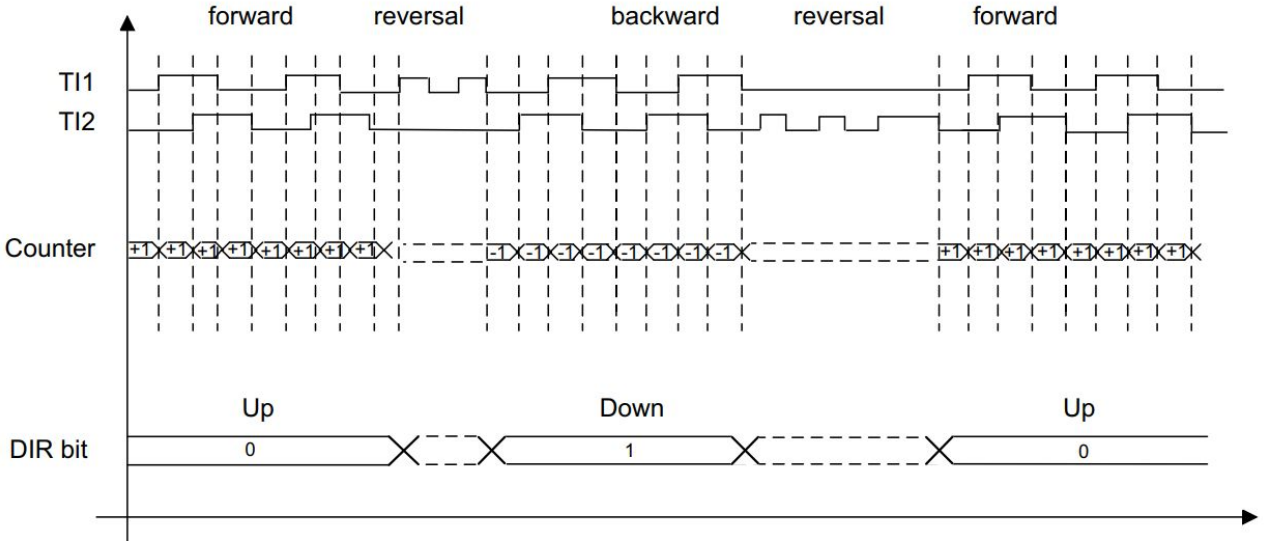


source: ST an4013

Counting resolution

Two resolutions:

- **4x**: count up or down by 1 at rising or falling edge of both TI1 and TI2 channels



source: ST an4013

Practical considerations #1

- Let's say we have an encoder at CPR resolution
- This means each full rotation we will have CPR periods of TI1 and TI2 square waves
- The net increase/decrease in one rotation of the CNT register will be: CPR x resolution
- Example: 1000 CPR encoder, QEI set to 2x resolution (rising/falling edges only of TI1)
  - Each rotation CNT increases/decreases by 2000

Practical consideration #2

- Recall we use timers to decode encoders!
- We can use the autoreload register (ARR) to generate interrupts for each full revolution (if we need!)
- Let's say we have a rotary incremental encoder with 1000 CPR at 4x resolution
- If we set ARR to 3999, an interrupt will be generated each time the counter overflows 3999
- Many encoders have an Index pin indicating a full revolution
- The index pin could be connected to an external interrupt input (EXTI) triggering a CNT reset in the ISR

Example

- We have a motor running at 100 rev/s (6000 rpm)
- We want to get an interrupt each time we sense a full revolution
- The encoder has a `CPR = X`
- We are using a QEI set to 4x resolution
- How should we set `ARR`?

```
ARR = 4X - 1
```

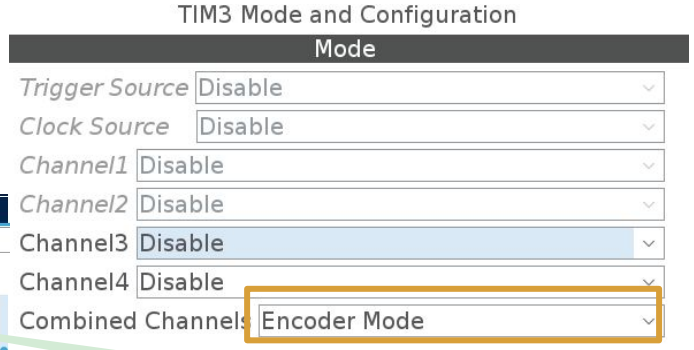
Example

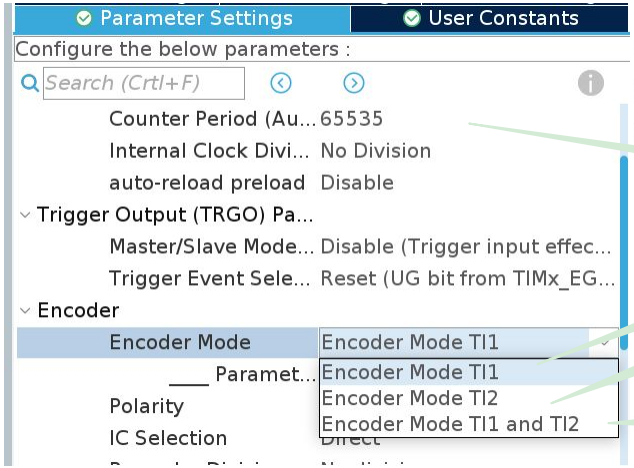
- You have a motor running at 9000 rpm (rev per minute)
- The encoder has a 1000 CPR and captured with 4x resolution
- You are using a 16bit timer set in interrupt mode (recall: interrupts generated when the CNT hits ARR)
- How many interrupts per second are generated using the largest possible ARR?
- What happens if you set ARR=999

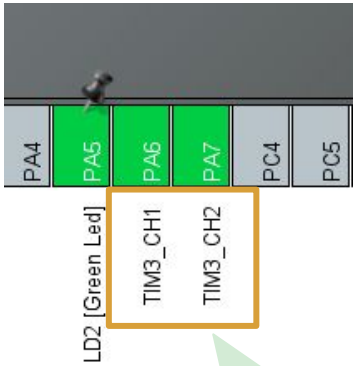
CubeMX Configuration

- Select any general purpose Timer
- In “Combined Channels” select “Encoder Mode”
- Configure resolution









ARR

res 2x

res 2x

res 4x

Pins to connect TI1 and TI2

Estimating the angular position

1. Zero the angle (i.e. physically put the shaft to angle = 0)
2. Reset `CNT` to a known value
3. Start the encoder:

   ```
   HAL_TIM_Encoder_Start(htimX, channels);
   ```

   channels: `TIM_CHANNEL_ALL, TIM_CHANNEL_1, TIM_CHANNEL_2`
4. Set ARR, let's call it `max_cnt`
5. Set the resolution (2x or 4x), let's call it `res`
6. Store the CPR in a variable, let's call it `cpr`
7. The current angular displacement in degrees is:

```
degs = (float)(TIMX->CNT)/((float)(res*cpr))*360.0
```

Estimate angular speed (basic)

```
#define CPR           ((uint32_t) 600)
#define RES           ((uint32_t) 4)
#define COUNTS_PER_REV  ((uint32_t)CPR*RES)

float estimate_speed_rad_s(void) {
   static uint16_t cnt_prev = 0;
    uint16_t cnt_now = TIMx->CNT;

    /* This only works if ARR = 0xFFFF (or 0xFFFFFFFF) */
    int16_t delta_cnt = (int16_t)(cnt_now - cnt_prev);

    cnt_prev = cnt_now;
    return ((float)delta_cnt * 2.0f * M_PI) /
            ((float)COUNTS_PER_REV * TS);

}
```

Angles are not real numbers they live in S^1

- Angles are not real numbers, they are elements of the circle S^1
- In fact, due to periodicity we know that $\theta = \theta + 2\pi$
- In time sampled systems velocity is computed as the difference quotient

$$\omega_k = \frac{\theta_k - \theta_{k-1}}{\Delta T}$$

- The issue is that if $\theta \in [0, 2\pi)$ when the angle "wraps-around" the difference between two angles shows a discontinuity!
- Example: $\theta_{k-1} = 359^\circ, \theta_k = 1^\circ \Rightarrow \Delta\theta_k = -358^\circ$
- However, the real net angular displacement on the circle is

$$\Delta\theta_k = 2^\circ$$

How to solve this? Circular difference using `atan2`

- Compute the naive angular difference $\delta\theta_k = \theta_k - \theta_{k-1}$
- Then use atan2 to map $\delta\theta_k$ to $[-\pi, \pi)$:

$$\Delta\theta_k = \mathrm{atan2}(\sin\delta\theta_k, \cos\delta\theta_k)$$

- Now, since $\Delta\theta_k \in [-\pi, \pi)$ we can use:

$$\omega_k = \frac{\Delta\theta_k}{\Delta T}$$

Implementation (using angles)

```
#include <math.h>
float estimate_omega_from_angle(float theta_now,
                                float theta_prev,
                                float Ts)
{
    float delta = theta_now - theta_prev;
    float delta_wrapped = atan2f(sinf(delta), cosf(delta));
    return delta_wrapped / Ts;
}
```

Implementation (using counts as inputs)

```
#include <math.h>
float estimate_omega_from_counts(uint32_t cnt_now, uint32_t cnt_prev,
                                 uint32_t counts_per_rev, float Ts) {
    int32_t delta = (int32_t)cnt_now - (int32_t)cnt_prev;
    if (delta > (int32_t)(counts_per_rev / 2))  {
        delta -= (int32_t)counts_per_rev;
    } else if (delta < -(int32_t)(counts_per_rev / 2))  {
        delta += (int32_t)counts_per_rev;
    }
    return ((float)delta * 2.0f * (float)M_PI) /
((float)counts_per_rev * Ts);
}
```

Estimate angular speed

- Estimating (low) angular speed can be problematic
- Low speeds mean few counts per sampling interval
- If we sample in the control loop the difference of CNT we can get high noise
- Recall CNT is an integer!
- If the shaft is rotating slow (or CPR is low) the CNT difference will be small => high quantization error!
- Some microcontrollers (not STM32) provide an estimation of the speed in hardware

One way to estimate the angular speed

Time-stamping algorithm [Merry et al., 2010]

- Capture via a high-resolution clock with sampling frequency f_s, the time instants and positions of a number of encoder events
- Perform at the controller's sampling rate $f_c << f_s$ an m-th-order polynomial fit through the aforementioned events to approximate the position of the shaft

$$x(t) = p_m t^m + p_{m-1} t^{m-1} + \ldots + p_0.$$

One way to estimate the angular speed

$$x(t) = p_m t^m + p_{m-1} t^{m-1} + \ldots + p_0.$$

- Let t_i and x_i denote the time instant and position corresponding to the i-th encoder sample
- k is the index of the most recent event
- We solve P (the coefficients of the best polynomial approx) from: $AP = B$

$$A = \begin{bmatrix} t_{k-n+1}^m & t_{k-n+1}^{m-1} & \cdots & t_{k-n+1} & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ t_k^m & t_k^{m-1} & \cdots & t_k & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} p_m & p_{m-1} & \cdots & p_1 & p_0 \end{bmatrix}^\top$$

$$B = \begin{bmatrix} x_{k-n+1} & \cdots & x_k \end{bmatrix}^\top$$

One way to estimate the angular speed

$$x(t) = p_m t^m + p_{m-1} t^{m-1} + \ldots + p_0.$$

$$A = \begin{bmatrix} t_{k-n+1}^m & t_{k-n+1}^{m-1} & \cdots & t_{k-n+1} & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ t_k^m & t_k^{m-1} & \cdots & t_k & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} p_m & p_{m-1} & \cdots & p_1 & p_0 \end{bmatrix}^\top$$

$$B = \begin{bmatrix} x_{k-n+1} & \cdots & x_k \end{bmatrix}^\top$$

- The least square solution can be obtained by using the pseudo-inverse: $P = (A^\top A)^{-1} A^\top B$
- Now we can compute the velocity since we have x(t), we can differentiate:

$$v(t) = \sum_{i=1}^{m} i p_i t^{i-1}$$

References

- Merry, R., van de Molengraft, M., and Steinbuch, M. (2010). Velocity and acceleration estimation for optical incremental encoders. Mechatronics, 20(1), 20–26. Special Issue on “Servo Control for Data Storage and Precision Systems”, from 17th IFAC World Congress 2008
- AN4013, Application note, STM32 cross-series timer overview